The Journal of Supercomputing (2024) 80:27034–27072
https://doi.org/10.1007/s11227-024-06431-z

A multi-objective approach for optimizing IoT applications offloading in fog–cloud environments with NSGA-II

Ibtissem Mokni[1,2] · Sonia Yassa[2]

Accepted: 5 August 2024 / Published online: 25 September 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

The Internet of Things (IoT) has become a pervasive phenomenon, with applications in a multitude of sectors, including healthcare, smart agriculture, smart cities, transportation, and water management. This has led to a significant generation of Big Data. In order to process this substantial volume of data efficiently, there is a pressing need for a platform capable of handling large quantities. However, real-time applications face challenges in cloud processing due to high latency. As a complementary infrastructure to the cloud, fog computing emerges as a viable solution by facilitating task processing, networking, and data storage in cloud data centers accessible to mobile users. The offloading of tasks represents a promising solution to the resource constraints inherent in IoT applications, particularly within the context of fog computing. This process entails the execution of particular components of mobile applications within a fog–cloud environment, to reduce execution time and energy consumption. The objective of our research is to optimize task offloading in IoT within heterogeneous environments, taking into account conflicting constraints. This optimization challenge is formulated as a multi-objective problem, with a particular focus on energy consumption and latency, as well as quality of service metrics. The proposed solution, TOF-NSGAII, is designed to respect the finite resources of fog computing, balancing workloads to meet the latency requirements of IoT tasks. The widely employed meta-heuristic, the non-dominated sorting genetic algorithm (NSGA-II), has been adapted to generate a set of non-dominated multi-objective task offloading optimization solutions, considering both energy consumption and latency. The experimental results demonstrate the efficacy of TOF-NSGAII in generating task offloading solutions that distribute executed tasks between fog and cloud computing environments in a judicious manner, based on their specific requirements. Furthermore, the generated non-dominated solutions demonstrate optimality in terms of energy consumption, with an average reduction of 12.18% compared to alternative approaches. It is noteworthy that our approach introduces only a marginal increase in latency, amounting to 0.38%, which can be considered negligible.

Extended author information available on the last page of the article

Keywords Offloading · Optimization · Fog computing · Cloud computing · Latency · Energy consumption · NSGA-II

1 Introduction

1.1 Context and issues

The Internet of Things (IoT) refers to the interconnection of everyday objects with digital infrastructures. By embedding computing and communication capabilities in objects, the IoT enables the development of intelligent services across various industries, including smart cities, Industry 4.0, and healthcare systems [11]. However, the widespread adoption of IoT technology has resulted in a significant increase in the generation of Big Data, underscoring the necessity for an effective platform to handle vast amounts of data.

The implementation of large-scale IoT environments is confronted with several challenges, including the heterogeneity of objects, their limited lifespan, and mobility, and the absence of platforms for the autonomous management of numerous objects and their associated services [25]. The management of IoT environments through existing cloud platforms is constrained by the sheer number of deployed objects, the necessity for real-time processing, and the requirement for privacy in certain applications [12, 32, 33]. Fog computing serves as a supplementary infrastructure to cloud computing, providing the capability to process tasks, manage networking, and store data in cloud data centers that are accessible to mobile users. It proves particularly valuable for applications and services that may not be well-suited for traditional cloud environments [26, 34]. Examples of such applications include real-time streaming, the Internet of Things (IoT), and any distributed application that demands low latency and high connectivity. The decision to offload tasks from IoT devices to either cloud or fog computing resources hinges on various factors, including the availability of resources, network bandwidth, and latency requirements. Although offloading tasks to remote cloud servers can alleviate the processing burden on IoT devices, it may introduce increased latency owing to data transmission time. In contrast, fog computing facilitates the offloading of tasks to nearby computing resources, such as edge servers or gateways. This proximity results in reduced latency, enhanced performance, and greater control over data processing and storage [19]. The choice between cloud computing and fog computing for task offloading is contingent upon the specific requirements of the application. Considerations include network connectivity, latency constraints, and the availability of resources. This decision-making process involves a careful evaluation of factors to optimize the overall performance and efficiency of the system in question [13].

1.2 Objectives

The objective of this paper is to achieve a balance between delay and energy consumption by jointly minimizing energy consumption and delay for computing

devices within a fog cloud computing framework. Task offloading represents a promising solution in the context of fog computing, offering a potential means of addressing resource limitations in the context of IoT applications. with the goal of improving execution time and reducing energy consumption by running part or all of a mobile application in a fog–cloud environment While cloud computing provides access to multiple application resources, it often results in longer response times for applications that require faster processing due to the distance from terminal equipment. To address this challenge, the collaboration between cloud and fog computing is seen as a solution. The key contributions of this work are to address the joint optimization of energy consumption and delay in computing devices within a fog cloud computing framework.

1.3 Contributions

The key contributions of this work are as follows:

- We express the joint optimization problem of energy consumption and time delay in the form of a multi-objective problem.
- Respect the limited capabilities of fog computing by balancing workloads between fog and cloud environments while adhering to predefined constraints.
- Given the NP-hard nature of the task offloading problem, we address the posed problem using a two-phase approach:
  - The first phase encompasses a preprocessing step, whereby tasks are classified according to their level of processing intensity, thereby achieving equilibrium between fog and cloud computing workloads.
  - The second phase of the approach focuses on the distribution of tasks within the system, which is achieved through the adaptation and implementation of the well-known meta-heuristic NSGA-II.
- To achieve a compromise between latency and energy consumption, we used the Pareto optimization approach, which provides a set of compromised solutions. This allows users to select the most suitable option based on their specific needs.

1.4 Paper organization

The remainder of this paper is organized as follows: Sect. 2 provides background information on key concepts such as the edge-fog–cloud environment and the collaboration between fog and cloud computing. Section 3 reviews related work on the offloading problem, discussing various methods including 3.1 approximate-based methods, 3.2 AI-based methods, and 3.3 hybrid-based methods. In Sect. 4, we present the problem formulation, system modeling, objectives, and algorithmic methods for resolution. Section 6 analyzes the results of our experiments and evaluates the performance as described in 6.2. Finally, Sect. 7 concludes the paper and discusses future trends.

2 Background

As illustrated in Fig. 1, the distinct architectural layers delineated in our work are as follows: the IoT applications, the edge layer, the fog layer, and the cloud layer. In the following sections, The hierarchical representation of these architectural levels focuses on the progression from the IoT application domain through the intermediate layers of edge computing and fog computing, culminating in the overall cloud infrastructure.

- **Cloud computing**: The concept of cloud computing represents a significant expansion in the availability of on-demand resources to users, who are able to reserve and release these resources according to their specific requirements. For the majority of users, the cloud represents a novel computing model that offers a source of economic efficiency, as it responds effectively to service demands on a pay-per-use basis. In contrast to traditional models, cloud users do not typically possess the underlying IT infrastructure, which is instead provided by cloud service providers. These providers offer a range of services, including infrastructure as a service (IaaS), platform as a service (PaaS), and software as a service (SaaS), among others. As this paradigm gains dominance in both industry and academia, a multitude of definitions have emerged [10, 14]. The definition provided by the National Institute of Standards and Technology (NIST) has been selected for use in this context. As outlined in reference [8], the essential aspects of cloud computing are as follows: "Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction." It can therefore be concluded that cloud computing

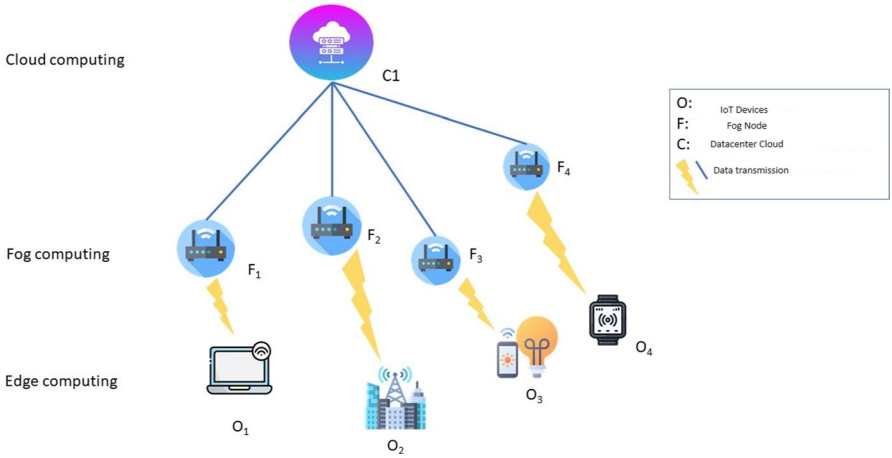

Fig. 1 System architecture

is an appropriate means of allocating various resources rapidly, easily, and efficiently.

- **Fog computing**: represents a decentralized computing infrastructure in which data, compute, storage, and applications are distributed in a manner that is both logical and efficient, situated between the data source and the cloud. This concept extends cloud computing to the edge of the network, thereby bringing the benefits and services of the cloud closer to the point of data generation and processing.
- **Edge computing**: The layer of the edge environment, which is the computer infrastructure nearest to the source of data generation or consumption, is found at the lowest level [9]. In addition to edge servers or gateways placed at the network's edge, it usually consists of edge devices including sensors, cellphones, routers, and Internet of Things gadgets [15]. This proximity to the data source allows for faster processing, lower latency, and more efficiency in data communication, which makes it appropriate for applications like autonomous systems, augmented reality, virtual reality, and the IoT that call for real-time or low-latency processing. Serving as a mediator between the data source and the cloud, the edge environment offers localized networking, storage, and processing capacities [29].

2.1 Fog and cloud computing: effective collaboration

The collaboration between fog and cloud computing for task offloading entails the utilization of both fog computing (or edge computing) and cloud computing to facilitate the efficient execution of tasks. Fog computing represents a distributed paradigm that brings computation and data storage closer to the data source, typically situated between edge devices and the cloud. This configuration provides a middle layer of processing, as evidenced in [18]. In contrast, edge computing refers to the processing of data directly on edge devices, such as sensors or Internet of Things (IoT) devices, at the network's periphery. Cloud computing, on the other hand, provides computing services over the internet, encompassing storage, networking, and applications. The combination of fog and cloud computing allows for task offloading that considers the distinctive characteristics of each environment, optimizing the processing and resource utilization across the entire spectrum from the edge to the cloud.

By combining fog and cloud computing, tasks can be offloaded in a manner that optimally utilizes the strengths of both environments. For instance, tasks requiring low-latency processing of sensor data can be executed on a nearby fog node, enhancing response times. Conversely, tasks demanding high-performance computing or extensive storage can be offloaded to run on the cloud. Collaborative fog and cloud computing also enable the offloading of computational tasks from the fog to the cloud, thereby reducing the load on the cloud and enhancing task execution efficiency. In this paper, fog computing serves as an intermediary layer between devices and the cloud. It performs pre-processing and filtering on data

before transmitting it to the cloud. This approach reduces the volume of data sent to the cloud, thereby minimizing response times and improving the overall workflow performance.

It's essential to highlight that effective collaboration between fog and cloud computing necessitates well-architected, secure, and reliable communication between the two layers. Additionally, a robust orchestration mechanism is vital for resource management and efficient task offloading.

The collaborative functioning of the fog and cloud layers is orchestrated to execute tasks requiring a combination of low-latency and high-performance computing. Examples include real-time monitoring, control, and analytics[36]. It's noteworthy that this architecture remains an active area of research, with various iterations and implementations existing based on specific use cases and requirements.

2.2 Task offloading in fog cloud computing

The fundamental problem of offloading is to decide which tasks to be offloaded to cloud computing in such a manner that respects all imposed constraints and optimizes predefined objectives. We examine some of the constraints and objectives that contribute to this decision.

- Resource-based constraints: when a task requires more computing power than the native device can provide, some tasks must be transferred to a more resource-rich device.
- Load balancing-based constraints: once a server/node has reached its capacity to perform tasks, additional tasks will need to be distributed among other servers/nodes.
- Latency-based constraints: tasks that require low-latency execution might need to be processed closer to the source, influencing the offloading decision.
- Energy consumption-based constraints: tasks may be offloaded to reduce the energy consumption of the native device, extending its operational lifetime.

Broadly speaking, the task offloading challenge within the fog–cloud system revolves around identifying a viable offload solution that optimizes one or more quality of service (QoS) metrics. These metrics encompass factors such as processing time, cost, and latency, among others. Some task-offloading solutions focus on enhancing a single metric, while others aim to optimize multiple metrics. Consequently, the tackled optimization problem can be categorized into two main groups: single-objective optimization and multi-objective optimization. That is the process of simultaneously optimizing two or more conflicting objectives. In the context of task offloading within fog–cloud systems, for example, the objective of minimizing latency may be in conflict with the objective of reducing energy consumption. In contrast to identifying a single optimal solution in a single objective optimization, the objective is to identify a set of solutions, objective is to identify a set of solutions that represent the best possible compromise among the objectives, thus allowing users to select the solution that best suits their specific needs. This set of solutions

is referred to as the Pareto front. A solution is deemed Pareto optimal if no alternative solution can enhance one objective without simultaneously impairing at least one other objective. This principle was named after the Italian economist Vilfredo Pareto, who observed that 80% of the land in Italy was owned by 20% of the population [7]. The Pareto principle, also known as the 80/20 rule, suggests that a few factors contribute to the majority of outcomes and has been widely applied to various fields such as business, economics, and personal development.

3 Related work

To this day, a large volume of research work has been devoted to offloading with the objective of jointly optimizing various QoS metrics. With different methods, the task offloading problem has been treated statically and dynamically in the literature as an optimization problem, a constraint satisfaction problem, and both. According to the existence, we classified these methods into three main categories:

- Approximate-based methods are techniques that involve approximating the execution of tasks on resource-constrained devices to reduce their computational requirements.
- AI-based methods involve the use of machine learning algorithms to optimize the offloading decision process. These methods aim to improve the performance of fog computing by enabling efficient allocation of tasks between fog nodes, edge devices, and cloud servers.
- Hybrid-based methods combine multiple approaches, such as optimization-based, and machine learning-based methods, to solve offloading problems.

The choice of an efficient method capable of producing an optimal solution in a reasonable computation time is a difficulty that every decision maker must face. A large volume of works on task offloading has been proposed in the literature. Let's quote some of them.

3.1 Approximate-based methods

The research presented in [27] introduces three constrained multi-objective evolutionary algorithms (CMOEAs) designed to tackle computational offloading challenges within collaborative edge and cloud networks in the IoT domain. The authors establish a constrained multi-objective computation offloading model that takes into account both time and energy consumption considerations in a mobile environment.

Inspired by the push and pull search framework, the authors have developed three constrained multi-objective evolutionary algorithms (CMOEAs) that take advantage of population-based search algorithms and incorporate adaptive constraint handling mechanisms.

Concerning this paper [41], the authors propose task offloading in a fog computing environment with the objectives of minimizing overhead and achieving a balance

between execution time and energy consumption. The authors use both genetic algorithm (GA) and particle swarm optimization (PSO) algorithms in their approach.

In the same way, in this work [5], a dynamic task offloading approach in fog computing has been introduced. The authors explore widely used population-based meta-heuristics to optimize the offloading process for a set of dynamic tasks using orthogonal frequency division multiplexing (OFDM) communication. They use a similar approach to address the challenges associated with this problem.

Furthermore, a parallel multi-threading framework is proposed in [2] as a means of generating an optimal offloading solution while selecting the most suitable subcarrier for each offloaded task. The authors adapted the ET-DTCO algorithm, which is an efficient method for offloading the computation of dependent tasks in a three-layer system. The algorithm considers two quality of service (QoS) parameters: energy consumption and task processing time. Moreover, the Firefly algorithm is adapted to optimize the selection and unloading of computation modes and to determine the optimal solution for executing tasks locally, offloading them to fog computing, or cloud computing, taking task dependencies into account. Conversely, the author of [16] puts forth an offloading system optimization approach with the objective of minimizing energy consumption, formulating task offloading as a multi-objective problem. The approach is twofold, aiming to reduce both the overall energy consumption of the system and the task execution time. This is achieved through the utilization of two meta-heuristic techniques: the NSGA-II and the Bees algorithm. Also, we found that the objective of the approach presented in [14] is to develop a task-offloading strategy that strikes a balance between energy consumption and latency, which is a critical consideration for IoT devices. The objective is to optimize energy consumption and latency as a multi-objective problem, employing the non-dominant sorting genetic algorithm (NSGA-II) and the Bees algorithm for resolution. In their paper [37], the authors present a framework called FRATO (fog resource-aware adaptive task offloading), which is designed to facilitate decision-making for offloading tasks in IoT–fog–cloud systems. The optimization problem of the FRATO framework is solved using a method based on particle swarm optimization (PSO). Furthermore, two distributed resource allocation mechanisms have been developed based on the optimized offloading solutions. These mechanisms are responsible for the allocation of fog resources for the execution of computing tasks. In addition, to optimize offloading from edge devices to the cloud, this article's [21] primary goal is to design an auction-based mechanism for dependent task offloading to enhance efficiency for multiple IoT users. To achieve this, the authors propose a truthful auction mechanism. This strategy considers both the structure of the task graph and the current status of the mobile edge computing (MEC) environment. Furthermore, they design a heuristic-dependent task assignment for the winners to improve the efficiency of task offloading. Also, in the work by Yadav et al.[40], the authors utilize a multi-objective optimization technique to address the issue of computation offloading in fog computing. This approach addresses the limitations of resource-restricted mobile devices by enabling task offloading to nearby fog nodes, thereby enhancing the quality of service and user experience. The authors employ the MOGWO technique for optimizing offloading decisions, marking the inaugural application of this method in fog computing. They enhance the MOGWO algorithm

by integrating a gravity reference point method to propose an enhanced version, E-MOGWO. The objective is to identify optimal offloading targets by considering parameters such as energy consumption and computational time in a heterogeneous, scalable environment with multiple fog nodes and users. On the other hand, This paper [30] addresses the challenge of executing compute-intensive mobile applications on resource-limited mobile devices by offloading tasks to edge servers. The issue of task offloading is formulated as an NP-hard problem, which indicates that the optimal solution is computationally complex to find. To address this challenge, the authors propose a metaheuristic-based task offloading mechanism utilizing the NSGA-II, designated as iNSGA-II. The article presents an innovative approach to the task offloading problem in edge–cloud environments through the employment of an enhanced genetic algorithm. The iNSGA-II mechanism effectively balances the computational load between mobile devices and edge servers, optimizing resource utilization and enhancing performance in terms of energy efficiency and execution time.

3.2 AI-based methods

The work presented in [9] proposes an optimized offloading mechanism based on classical multi-armed bandit (MAB) theory. This mechanism enables unmanned surface vehicles (USV) cluster nodes to make dynamic offloading decisions based on the potential computational performance of neighboring team nodes, thereby minimizing the average offloading time of computational tasks. The optimized algorithm, called the adaptive upper confidence boundary (AUCB) algorithm, is evaluated through simulations.

Similarly, the author of [39] uses the combinatorial multi-armed bandits (CMAB) framework to construct an online learning-based task offloading algorithm for delay-sensitive applications in dynamic fog networks. This work first proposes a resource-sharing algorithm to negotiate the capacity of each fog node, aiming to minimize the latency of task offloading by jointly optimizing the task allocation decision and spectrum schedules.

Moreover, in [38], the authors propose a task offloading mechanism in a vehicular fog computing environment, employing a combination of on-policy reinforcement learning and fuzzy logic. The primary objective is to develop an energy-efficient scheduling service for the allocation of tasks to fog vehicles.

Additionally, in [28] a task offloading approach is proposed in the context of Module Placement by Classification and Regression Tree Algorithm (MPCA). The authors select the best fog computing device for the modules by MPCA. In the first step, the energy consumption of the mobile devices is checked. If this value is higher than the energy consumption of Wi-Fi, the offloading is performed. Also, the challenge of this paper [1] is to perform complex tasks such as machine learning with minimal power consumption. This talk presents a three-tiered IoT–fog–cloud model and evaluates it using real-world datasets in three scenarios: fog-only, cloud-only, and fog–cloud collaboration. However, This paper [6] addresses the challenge of joint task offloading and resource allocation in multi-fog node systems, which handle heterogeneous service tasks. The problem is modeled as a partially observable

stochastic game, where each fog node aims to maximize its local rewards while dealing with the limitations of local observations. To overcome the issue of partial observability, the authors propose the use of a deep recurrent Q-network (DRQN) to estimate optimal value functions. The effectiveness of the proposed solution is evaluated through a comparative analysis with deep Q-networks (DQN) and deep convolutional Q-networks (DCQN). The results demonstrate the relative advantages of using DRQN in this context, providing insights into the benefits of different neural network approaches for task offloading and resource allocation in complex, dynamic fog computing environments. Moreover, the article in [3] resents a multi-classifier-based algorithm designed to enhance energy efficiency in the context of task offloading within fog computing environments. The proposed algorithm employs a multi-classifier approach to facilitate informed decision-making regarding the offloading of tasks to fog nodes, with the overarching objective of reducing overall energy consumption. By evaluating the computational requirements and current load of fog nodes, the algorithm selects the most suitable nodes for task execution on a dynamic basis. The effectiveness of this approach is demonstrated through comprehensive simulations, which show significant improvements in energy efficiency compared to traditional task-offloading methods. This multi-classifier strategy provides a robust solution for optimizing resource utilization and minimizing energy usage in fog computing systems.

Similarly, this work [35] explores the practice of distributing computational tasks across multiple nodes within a fog computing network. This approach is particularly relevant in scenarios where the execution of tasks can be made more efficient by utilizing the resources available at various locations within the network. The primary objective of this research is to examine and propose collaborative strategies for optimizing the distributed task offloading process in Fog Computing networks. In this context, "cooperation" refers to collaborative efforts among nodes or entities within the network to enhance task execution. The proposed solution is based on the exploration and development of cooperative mechanisms, which demonstrates the authors' investigation into how nodes in the Fog Computing network can collaborate synergistically to improve the overall efficiency and effectiveness of task offloading.

Furthermore, the authors of this work [22] address the issue of task graph offloading in Mobile Edge Computing (MEC) by developing a solution based on the Deep Reinforcement Learning (DRL) algorithm and Markov decision process (MDP). The proposed solution aims to enhance the user experience by optimizing task scheduling. This paper primarily addresses the issue of task graph offloading. The objective is to address the challenge of offloading tasks in an MEC environment, enhance the user experience through efficient task scheduling, and ensure the solution is adaptable to environmental changes.

3.3 Hybrid-based methods

In [15], an offloading approach is proposed for determining the optimal location for executing modules, which could be on mobile device, fog nodes, or cloud

servers. This approach combining the use of a greedy technique that prioritizes sibling nodes, followed by parent and sibling nodes and a deep reinforcement learning (DRL) algorithm to identify the optimal destination for running the module, which creates a tradeoff between energy consumption and module cost.

In [4] a task offloading approach is proposed based on a reinforcement-learning-based state-action-reward-state-action (RL-SARSA) algorithm to solve the resource management problem in the edge server, and make the optimal offloading decision to minimize the system cost, including power consumption and computation time. The main objective of [24] is to design a computation offloading strategy for each of the individual tasks to minimize the weighted sum of the delay and energy consumption in UAV-assisted mobile edge computing systems (MEC). The author of this paper is focused on designing a distributed deep neural network (DNN) model to find the optimal offloading strategy, he generates the dataset by applying the semi-definite relaxation (SDR) to the quadratically constrained linear program (QCLP) to train the DNN and considers three DNNs that are trained in the same training period for the distribution.

We present the relevant works in Table 1.

3.4 Discussion

From the task-offloading research works we have cited, we argue that the task-offloading problem has been widely addressed with different methods, techniques, objectives and environments.

It is worth noting that the majority of the works studied have employed an approximate method to deal with this problem. This method includes a genetic algorithm, particle swarm optimization algorithm, and so on. In Fig. 2, we illustrate this difference.

Consistent with the previous points, we are particularly interested in the importance of the task offloading problem and its crucial impact on the performance of fog and cloud environments. Furthermore, we are motivated by approximate methods to propose an optimal task offloading approach that optimizes latency and energy consumption.

The NSGA-II algorithm, renowned for its resilience and efficacy in addressing multi-objective optimization challenges, offers substantial benefits in the context of task offloading in fog computing environments for Internet of Things (IoT) applications. By effectively balancing multiple objectives, including minimizing energy consumption and maximizing computational resource utilization, the NSGA-II algorithm ensures optimal task allocation to fog nodes. The innovation of our approach lies in the incorporation of a pre-processing phase, which assesses the computational capacity of fog nodes and ranks tasks based on the current load and capacity before distribution. This pre-processing step, in conjunction with the NSGA-II task distribution algorithm, enables a more efficient task offloading solution and markedly reduces energy consumption, particularly as the number of tasks increases.

In the following section, we will present the modeling of our approach of task offloading in the fog cloud computing environment.

Table 1 comparison table of related works

Ref	Objectives	QoS	Algorithm	Environment
[15]	Tradeoff between energy consumption and module cost —The best place to execute the modules (Fog/Cloud)	EC; D L	DRL+greedy technique	Cloud–fog
[41]	Minimization of the overhead of offloading tasks to a fog computing environment -compromise between ET and EC	ET; EC; D	Particle swarm optimization; genetic algorithm	Fog computing
[27]	Solve IoT-related computational offload problems in collaborative edge and cloud networks Optimize EC and Latency	L; ET; EC	Proposed algorithms (push and pull)	Edge–-fog–cloud
[31]	Optimizes the task offloading decision Examining the trade-off between task PT and EC	EC;D	Particle swarm optimization; genetic algorithm	Fog computing
[5]	Dynamic task offloading in fog computing Trade-off between the delay and energy consumption	EC;D	OFDM; particle swarm optimization	Edge–fog computing
[2]	Provides an efficient offloading strategy	EC; D;ET	Firefly algorithm	Edge–fog–cloud
[40]	Computation offloading in fog computing	EC, computational time	MOGWO	Fog–cloud
[16]	Ensure an unloading strategy with a focus on reducing the total system energy consumption and response delay	EC;D	NSGA-II Bees algorithm	Fog–cloud computing
[30]	Executing compute-intensive mobile applications on resource-limited mobile devices by offloading tasks to edge servers	EC, Execution time	NSGA-II	Edge–cloud computing
[9]	Propose an optimized offloading mechanism based on the classic multi-armed bandit (MAB) theory - Make a dynamically offloading decisions	D	AUCB algorithm	Fog–cloud computing
[20]	This article adopts a strategy to balance energy consumption and Daley	EC; D	offloading algorithm STML	Fog–cloud
[14]	Make a trade-off between energy consumption and latency	EC, D	NSGA-II Bees algorithm	Fog–cloud computing
[24]	The task offloading strategy with the objective of minimizing the overall cost in terms of weighted delay and energy consumption	EC, D	Deep neural network	MEC server + edge
[6]	Studies the design of a joint task offloading and resource allocation control for heterogeneous service tasks	memory, TP;	Deep Q-learning	Fog - cloud computing

Table 1 (continued)

Ref	Objectives	QoS	Algorithm	Environment
[37]	Obtain a set of effective offloading strategies including optimal and sub-optimal solutions for mapping -provide IoT services with minimized context-sensitive delay	D, ET	PSO-based algorithm, TPRA, MaxRU	Fog–cloud computing
[28]	Select the best FDs (Fog devices) The trade-off between time and energy	EC, D	MPCA: classification and regression tree	Fog computing
[39]	Propose an online learning-based task offloading algorithm for delay-sensitive applications in dynamic fog networks	Cost, Latency, resources	Multi-armed bandits	Fog computing
[1]	-Distribution and execution of tasks, with achieving high scalability of IoT services and also managing energy consumption	ET, EC	Smart global gateway, FCFS	Fog–cloud computing
[4]	Make the optimal offloading decision for minimizing system cost -Propose a reinforcement learning-based state-action-reward-state-action (RL-SARSA) algorithm to resolve the resource management problem in the edge server	C, EC,D	RL-SARSA algorithm	Edge–cloud computing
[3]	-To enhance energy efficiency in the offloading of tasks -Decentralized computing resources in fog computing closer to the edge of the network	E, D	Multi-classifiers	Edge–fog computing
[21]	Improve the efficiency of task offloading for multiple IoT users	L, makespans	Heuristic dependent	Edge–cloud computing
[22]	A methodology for the offloading of task graphs in a mobile edge environment is presented	Makespan, deadline	MDP, DRL	Edge computing

PT processing time, *GA* genetic algorithm, *OFDM* orthogonal frequency division multiplexing, *ET* execution time, *D* delay, *L* latency, *EC* energy consumption, *TOF* task offloading, *AUCB* adaptive upper confidence boundary, *DNN* deep neural network, *TP* transmission power, module placement method by MPCA: classification and regression tree algorithm, *RL-SARSA* reinforcement learning-based state-action-reward-state-action, *TPRA* task priority-based resource allocation, *MaxRU* maximal resource utilization-based allocation, *DRL* deep reinforcement learning, *MDP* Markov decision process

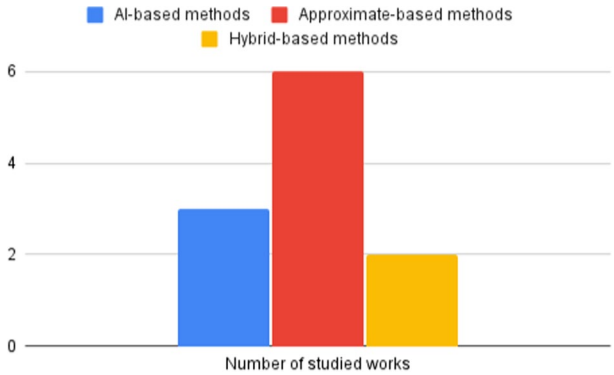

Fig. 2 Comparison of studied works methods

4 System model and Problem formulation

4.1 System and application model

As illustrated by the system architecture in Fig. 1, the system is composed of three layers: the cloud computing layer, the fog layer, and the IoT layer. The cloud computing layer consists of servers. The fog layer is composed of fog nodes (F), with each node consisting of a set of virtual machines (VMs). The IoT layer comprises IoT objects (O). Each IoT object (O_i) generates a set of tasks (T) and can only be connected to one fog node (F) via a base station (BS).

IoT objects make requests to execute their tasks and forward them to the fog nodes. If the resources of the nodes are not sufficient to execute these requests, a collaboration with the cloud is established. The notations used in this research are shown in Table 2.

Each IoT object O is defined by a vector $Vo = [ido, Io, So, Co, AP]$, where:

- ido: the identifier of the object O,
- Io: the spatial location of the object,
- So: the type of the IoT object,
- Co: material capacity,
- AP: the computer application realized by the object.

This vector encapsulates key properties of each IoT object, such as its unique ID, location, type, hardware capacity, and the application it runs.

Each fog node $F_i \in F$ is defined by a vector $VF = [idf, Pf, Tf, b, CSF, CCF]$, where:

- idf: the identifier of fog node,
- Pf: geographical position of the fog node,
- Tf: the set of all tasks executed in the fog node,
- b: set of virtual machines (VMs) of the fog node,

Table 2 System rating table

Symbol	Definition
IDf	Identifier of a fog node
Pf	Geographical position of a fog node
b	Set of VMs of a fog node
CSF	Storage capacity for a fog node GigaByte
CCF	Computational capacity of a fog node
CS	Storage capacity of one VM (Kb)
Cc	Computational capacity of a VM
IDC	Identifying a cloud data center
PDC	Geographic position
SDC	Server package
AP	IoT application
T_i	An IoT application task
CT	Load of an IoT task
Din_{Ti}	Input data volume of a task in kbit
$Dout_{Ti}$	Job output in kbps
TE_{Ti}	Time of entry to the processor (seconds)
TF_{Ti}	End time of the execution (seconds)
TT_{Ti}	Transmission time of a task (seconds)
PT_{Ti}	Propagation time of a signal through a media (m.s)
Latency	Data transfer time via network (milliseconds)
W	Bandwidth that connects an IoT object with a VM (Megabit per second)
V	Speed in meters per second to send a job
DS	Distance in meters between an IoT object and a virtual machine
TEX_{Ti}	Processing time for task execution in second(s)
FU	Processor usage frequency (Gigahertz)
E^{o}_{Ti}	Electrical energy consumption for a task executed at the IoT object level (Joule)
EN^{vmj}_{Ti}	Energy consumption for a task executed at the virtual machine level (watts)
ETT_{Ti}	Energy consumed by a task in transmission phase (Joule)
ETC_{Ti}	Energy consumed by a task during processing (Joule)
P_o	Energy of the terminal (joule)
Ps	Power of a fog/cloud node server (watts)

- *CSF*: storage capacity of the fog node calculated by Eq. 1.

This vector captures essential attributes of each fog node, including its ID, location, the tasks it handles, its VMs, and its total storage and computational capacities.

$$\mathrm{CSF} = \sum_{j=1}^{b} \mathrm{CS}_{vmj} \tag{1}$$

where CS_{vmj} is the storage capacity of each virtual machine VM_j within the fog node. The total storage capacity of the fog node is the sum of the storage capacities of all its VMs.

- CCF: computational capacity of the fog node calculated by Eq. 2.

$$CCF = \sum_{j=1}^{b} Cc_{vmj} \tag{2}$$

where Cc_{vmj} is the computing capacity of each virtual machine VM_j within the fog node. The total computational capacity of the fog node is the sum of the computing capacities of all its VMs.

Each virtual machine VM_j is defined by a vector Vvm = [idvm, CS, Cc], where:

- *idvm*: the virtual machine ID,
- *CS*: storage capacity of the VM,
- *Cc*: computational capacity of the VM.

This vector describes each VM by its ID, storage capacity, and computational capacity. Each cloud data center is defined by a vector VDC = [IDC, PDC, SDC] where:

- *IDC*: the identifier of cloud data center,
- *PDC*: geographical location,
- *SDC*: server package.

This vector includes the ID, location, and server configuration of the cloud data center.

Each server within the cloud data center consists of a set of interconnected virtual machines, facilitating the collaborative processing and storage of tasks when fog nodes require additional resources.

At a specific time period *i*, the fog computing layer includes multiple IoT applications, each of which is made up of a series of tasks. The area responsible for facilitating the efficient distribution of tasks between fog nodes and cloud servers is called a broker. Our objective is to achieve this distribution while simultaneously optimizing energy consumption during task execution and minimizing latency.

4.2 QoS metrics

4.2.1 Latency

The latency metric being adopted represents the time it takes to transfer data through the network, it can be measured as the sum of the request transmission time TT shown in Eq. 4 and the propagation time PT calculated in Eq. 5. PT is the time

it takes for a signal to propagate from one point to another across a medium. The latency is calculated in Eq. 3.

$$\text{latency} = \text{TT}_{T_i} + \text{PT}_{T_i};\ \forall T_i \in T \tag{3}$$

where

$$\text{TT}_{T_i} = \left(\frac{\text{Din}_{T_i}}{W}\right) \tag{4}$$

and

$$\text{PT}_{T_i} = \left(\frac{\text{DS}}{V}\right) \tag{5}$$

where:

- Din: is the input data volume of the task T_i.
- W: is the bandwidth that connects an IoT object with a VM_j.
- DS: is the distance in meters between an IoT object and a virtual machine.
- V: is the speed in meters per second of a VM to transfer a task.

Propagation time (PT) is the time taken for a signal to travel from one point to another across a medium, such as a wire, fiber optic cable, or through the air. It depends on the distance between the sender and the receiver and the speed of the signal in the medium. The propagation time measurement depends on the distance between each IoT object and the VM. To calculate this distance, we use the formula from Cartesian geometry for the distance between two points $A(x1, y1)$ and $B(x2, y2)$. Equation 6 presents the formula for this distance.

$$D(A, B) = \sqrt{((x_2 - x_1)^2 - (y_2 - y_1)^2)} \tag{6}$$

Execution time refers to the amount of time required to complete a given task. Total run time, on the other hand, is determined by measuring the time elapsed from when a task enters the processor (TE_{T_i}) until its execution is completed (TF_{T_i}), including the transmission time of the task request. Transmission time (TT) is the time required to send the data from the sender to the receiver, and it depends on the size of the data and the bandwidth of the communication channel.

$$\text{TEX}_{T_i} = \text{TF}_{T_i} - \text{TE}_{T_i} \tag{7}$$

4.2.2 Energy consumption

In the realm of IoT devices, each task can be executed either locally on the endpoint or transferred to a virtual machine in the fog or cloud. Consequently, it becomes imperative to examine power consumption metrics at two distinct levels.

- At the local level, we make the assumption that an endpoint's node is powered by an electric battery. In this scenario, the energy consumption can be calculated by multiplying the time required for the processor to complete a task (determined by the frequency of use Eq. 9) with the electrical power of the endpoint, denoted by P and measured in watts.

$$E_T^o = \mathrm{FU} \ * P_o \tag{8}$$

Where

$$\mathrm{FU} \ = \mathrm{TF}_{T_i} - \mathrm{TE}_{T_i} \tag{9}$$

- At the fog/cloud node, a virtual machine (VM) represents a fully virtualized environment operating on a physical server, complete with distinct characteristics like an operating system (OS) and processor. In our approach, the energy consumption formula for a given VMj is defined as the sum of the energy consumed during the transmission phase ETT_{T_i} and the energy consumed during the processing phase ETC_{T_i} of a task T_i.

$$\mathrm{EN}_{T_i}^{vm,F} = \mathrm{ETT}_{T_i}^F + \mathrm{ETC}_{T_i} \tag{10}$$

$$\mathrm{EN}_{T_i}^{vm,C} = \mathrm{ETT}_{T_i}^f + \mathrm{ETT}_f^C + \mathrm{ETC}_{T_i} \tag{11}$$

where

$$\mathrm{ETT}_{T_i} = \sum_{T_i=1}^{n} P_{T_i}^{\mathrm{Tra}} * \mathrm{TT}_{T_i} \tag{12}$$

and

$$\mathrm{ETC}_{T_i} = \sum_{Ti=1}^{\mathrm{nPs}} * \mathrm{TEX}_{Ti} \tag{13}$$

where $\mathrm{P}_{Ti}^{\mathrm{TRA}}$ defines the power of the sensor of an IoT object. Ps defines the power of a server of the fog/cloud node where VMs are located.

4.3 Multi-objective task offloading problem formulation

Task offloading involves in determining which tasks should be processed in the fog computing environment and which should be processed in the cloud, with the goal of providing a service that minimizes total energy consumption and maintains a delay no greater than a given threshold. This problem can be expressed as a binary integer programming problem with constraints. The optimization problem **OPT** of task offloading in fog computing is expressed by the formula 14. In **OPT**, the

objective is to minimize two functions: $f1$, representing the energy consumption E_{tt} due to task processing and data transmission in both fog and cloud environments, and $f2$, representing the data transmission time TR.

$$\mathrm{OPT} = \min(f1 : E_{tt}, f2 : TR) \tag{14}$$

- **Offloading objectives** The task offloading process starts when a request to execute a task $T_i \in T$, processed by an application AP_i of an IoT object O_i, is transferred to a fog node $\mathrm{F}_j \in \mathrm{F}$. The energy consumed during this process is divided into two parts, namely, the energy consumed to transmit the data through the BS base station which is the intermediate between IoT object layer and fog nodes, and the energy consumed during execution. The energy consumed to perform a task within fog node is denoted by $\mathrm{EN}_t^{vm,f}$. In the case where local fog resources are insufficient, the task is offloaded to a cloud server $\mathrm{S}_j \in \mathrm{SDC}$. We denote the energy consumption in this case by $\mathrm{EN}_t^{vm,c}$. In our work, we consider that the energy consumed during offloading must not exceed a predefined consumption threshold. We define in Eq. 15 the global energy of the fog and cloud computing system as the sum of the energy consumed during execution and the energy of transmission.

$$f1 : E_{\mathrm{TT}} = \sum_{T_i=1}^{n} \mathrm{EN}_{T_i}^{vm,f} * y^f + \sum_{T_i=1}^{n} \mathrm{EN}_{T_i}^{vm,c} * y^c \tag{15}$$

  where, the variables y^f and y^c are binary decision variables used to indicate which instance will process the request for object O. If the task is being forwarded to a fog node, the variable y^f will be assigned the value 1, while the variable y^c will be assigned the value 0. On the other hand, if the task is being processed at cloud data center, the variable y^c will be assigned the value 1 and y^f will be assigned the value 0. The second objective of task offloading considered in this work is latency minimization. In fact, the latency is the communication delay in exchanges between the object and the system. It is also known as delay or travel time. In absolute terms, we seek to achieve the lowest possible latency. Equation 16 is considered to identify and monitor the overall latency of our system.

$$f2 : \mathrm{TR} = \sum_{T_i=1}^{n} \mathrm{latence} * y^f + \sum_{T_i=1}^{n} \mathrm{latence} * y^c \tag{16}$$

  Notice that, y^f and y^c are binary decision variables to indicate on which instance the request of object O will be processed. If the task is forwarded to a fog node, the variable y^f takes 1 and the variable y^c takes 0. Otherwise, the variable y^c takes 1 and y^f takes 0.
- **Offloading constraints** The initial offload validity constraint within the system guarantees that each task is transferred to a single instance, either a fog node or a cloud data center. Formula 17 illustrates this condition, where y_{T_i}

represents the variable indicating that a task has been offloaded to a node. It is imperative that each task is transferred to a single node, either fog or cloud.

$$\mathrm{VD} = \sum_{vm_j=1}^{n} y_{T_i} \leq 1 \tag{17}$$

The second requirement is the deadline that the user predefines to not exceed. In this case, the condition is based on the latency metric which must be strictly lower than the deadline of a task. In Eq. 18 we present this condition.

$$\mathrm{TR} \leq \mathrm{Deadline} \tag{18}$$

The third constraint to validate offloading is an energy consumption threshold that must not be exceeded. A generated offload solution must have a lower energy consumption than the maximum power of a fog node or a cloud server which is defined by MAXE. This constraint is defined in Eq. 19.

$$E_{tt} \leq \mathrm{MAXE} \tag{19}$$

Taking this a step further, we assume that the usability of a fog node is limited, denoted by Eq. 20.

$$\mathrm{CCF} = \sum_{0}^{b} \mathrm{Cc}_{vmj} \tag{20}$$

CCF is the total capacity of a set of virtual machines in each node. Therefore, the execution capacity of a set of tasks in F must not exceed its computational resources. Subsequently, at the fog level, we performed a step of workload evaluation of each task. This step is developed to differentiate the tasks that can be executed on the fog layer and that are characterized by workloads within the capabilities of the available resources, and computationally intensive tasks that exceed the capacity of the fog. The evaluation is defined by Eq. 21 [23], which denotes the cost of executing a task CS divided by the capacity of a fog node CCF.

$$\mathrm{EC} = \frac{\mathrm{CS}}{\mathrm{CCF}} \tag{21}$$

The EC equation yields two groups of values, the light task group generates EC values tending toward 0 and less than 1, and the intensive task group generates EC values above 1.

$$y_{Ti} = \begin{cases} 1 & \text{if } 0 \leq \mathrm{EC} \leq 1 \\ 0 & \text{if } \mathrm{EC} \geq 1 \end{cases} \tag{22}$$

In case when the fog is over capacity, a collaboration with the cloud is established through the tasks offloading.

We illustrate the task offloading solution by Eq. 23. where we consider two decision variables, y^f is the fog decision variable, and y^c is the cloud decision variable. We assume that both variables are initialized to zero.

$$\begin{cases} y^f = 1, \text{ the task can be executed in fog,} \\ y^c = 1, \text{ the task is offloaded to cloud} \end{cases} \tag{23}$$

5 Proposed Task of floading solution with NSGAII algorithm (TOF-NSGAII)

The approach **T**ask **OF** floading with NSGAII algorithm (TOF-NSGAII) is proposed to create a strategy for offloading IoT tasks in a fog cloud environment while optimizing QoS metrics. In this respect, we are interested in developing the NSGA-II meta-heuristic, since it is well known in the literature as a sufficient solution to solve optimization problems. The different steps proposed in our TOF-NSGAII approach are the preprocessing step which consists of not overusing the limited fog resources and the second step the task distribution step which applies the adaptive NSGA-II algorithm for task offloading.

In the context of our research, NSGA-II plays a pivotal role in ensuring an effective equilibrium between offloading tasks to fog nodes and cloud servers.

The choice of NSGA-II for the proposed approach is driven by its robustness, efficiency in handling multi-objective optimization, and suitability for dynamic and complex environments. These attributes serve to demonstrate that NSGA-II is the optimal choice for optimizing task offloading in fog and cloud computing.

Step 1: Pre-processing:

The approach begins at instant t, where the fog computing layer has n tasks to be executed within a predefined time frame. At this moment, the fog layer broker starts by classifying the tasks into two categories, light and intensive, using the execution cost computation defined in Eq. 21. The broker, located in the fog layer, is responsible for distributing the tasks between the fog nodes and the cloud server. It utilizes an optimization algorithm, which also runs in the fog layer, to determine the optimal distribution of tasks. This ensures efficient use of resources and adherence to the time constraints for task execution.

In the algorithm 1, we introduce the classification step that takes as input a set of tasks T and the fog node where the tasks are located. The task classification algorithm calculates for each task the execution cost and initializes the variable y according to the result of the calculation. As an output, the tasks are classified into two classes according to the value of the variable y of each task.

Algorithm 1 Classification of tasks

$T = \{T_1, \ldots, T_n\}$ (tasks), fog computing node
$T = \{T_1, \ldots, T_n\}$ classified
$i \leftarrow 1$
for *each* T_i *in* T **do**
Calculate $EC_i = \frac{CS_i}{CCF_i}$
if $EC_i \leq 1$ **then**
$Y_i \leftarrow 1$
else
$Y_i \leftarrow 0$

Step 2: Task distribution (NSGA-II):

The task distribution process involves classifying tasks into fog and cloud computing environments. To achieve this, we adapted the NSGA-II algorithm proposed by [10], which is a powerful meta-heuristic method for solving multi-objective problems. The NSGA-II algorithm uses an elitist strategy and a diversity preservation mechanism, where the population of parents and children are merged and classified into several Pareto fronts. The next generation's population is then formed by selecting solutions from these Pareto fronts, starting with the first one. The crowding distance is used to determine the average distance between solutions and their direct neighbors in the objective space. The NSGA-II algorithm can be described in six basic steps, including initialization of the population, non-dominant sorting, crowding distance assignment, selection of individuals, genetic operators, and recombination and selection. The algorithm is applied iteratively until the population size exceeds the current population size.

To adapt the NSGA-II algorithm to our task distribution problem, a suitable solution structure and genetic operators need to be defined.

- **Solution Coding**

The NSGAII algorithm is composed of several generations (*NG*), each containing a set of populations. A population in *NG* is formed by a set of individuals *I* where each of them represents a feasible solution to the task offloading problem. A feasible solution by the Task-VM assignment is presented by IDT (the unique identifier of the task) and IDvm (the unique identifier of the virtual machine). Suppose we have a set of six tasks [T_1, T_2,..., T_6], a fog node with a set of virtual machines [VM^f_1,, VM^f_2, VM^f_3,..., VM^f_j], and a cloud data center with a set of virtual machines [VM^c_1,, VM^c_2, VM^c_3,..., VM^c_k]. Here is an example of how these tasks can be distributed over a set of virtual machines in the fog and cloud computing environments:

- Task T_1 is assigned to virtual machine VM_1 in the cloud (VM^c_1).
- Task T_2 is assigned to virtual machine VM_2 in the fog (VM^f_2).
- Task T_3 is assigned to virtual machine VM_1 in the fog (VM^f_1).

- Task T_4 is assigned to virtual machine VM_2 in the cloud (VM_2^c).
- Task T_5 is assigned to virtual machine VM_3 in the cloud (VM_3^c).
- Task T_6 is assigned to virtual machine VM_4 in the fog (VM_3^f).

The final distribution of tasks is presented in Table 3. This table defines the identifier of each task and the corresponding virtual machine to which it is assigned. By offloading tasks to either fog or cloud VMs, we aim to optimize the overall system performance in terms of latency and energy consumption.

Algorithm 2 NSGA-II for IoT task offloading

Input: Pt: Parent population, NG: Number of generations, T: Tasks, NF: Number of fog VMs, NC: Number of cloud VMs, $\mathrm{EN}_T^{vm,f}$: Energy consumed by a fog VM, $\mathrm{EN}_T^{vm,c}$: Energy consumed by a cloud VM, $\mathrm{L}^{vm,f}$: Latency of a fog VM, $\mathrm{L}^{vm,c}$: Latency of a cloud VM, Cc: Computational capacity of a cloud VM, CC_f: Computational capacity of a fog VM, *Deadline*: Task deadline, $MAXE$: Energy consumption threshold.
Output: Non-dominated solutions (Pareto front)

```
Initialize Population (Pt)
 while NG > 0 do
   while Population Size NP > 0 do
     for each individual I in the population (Pt) do
       Evaluate (Individual I)

     Select Parent1 ⟵ selection (Pt)
      Select Parent2 ⟵ selection (Pt)
      Perform Crossover (Parent1, Parent2)
      Apply Mutation (Parent1, Parent2)
      Qt ⟵ Generate Child Population

     NP ⟵ NP − 1
   Combine Rt ⟵ Pt ∪ Qt
    Perform Pareto Fast Non-Dominated Sort (Rt)
    Calculate Crowding Distance (Rt)
    NG ⟵ NG − 1
```

- **Pareto front:** In this context, the Pareto principle [10] can be applied by recognizing that the non-dominated solutions obtained by the algorithm represent the Pareto optimal solutions. These solutions correspond to the small number of inputs or factors that account for the majority of results or outcomes, as defined by the Pareto principle. By using the NSGA-II algorithm to find the Pareto optimal solutions, one can identify the most important trade-offs and factors that need to be considered when making decisions about IoT task offloading. In the initialization phase of the algorithm 3, for each task offloading solution SP_p in R_t, the domination counter n_p is set to 0 and the set S_p of solutions dominated by SP_p is initialized as empty. During the comparison of solu-

tions, for each pair of solutions SP_p and SP_q in R_t, if SP_p dominates SP_q (denoted as $SP_p \prec SP_q$), SP_q is added to the set S_p. Conversely, if SP_q dominates SP_p (denoted as $SP_q \prec SP_p$), the domination counter n_p is incremented. For identifying the first Pareto front, if a solution SP_p is not dominated by any other solution (i.e., $n_p == 0$), it is assigned to the first Pareto front (F_1) and its rank is set to 1. The construction of subsequent Pareto fronts begins by initializing a front counter i to 1. As long as the current front F_i is not empty, an empty set H is created to store the members of the next front. For each solution SP_p in the current front F_i, each solution SP_q in the set of solutions dominated by SP_p (S_p) has its domination counter n_q decremented. If n_q becomes 0, this means that SP_q belongs to the next front; its rank is set to $i + 1$ and it is added to H. The front counter i is then incremented, and F_i is updated to the set H. This process continues until no more solutions remain to be assigned to a new front.

Algorithm 3 Pareto (fast non-dominated sort) for task offloading in edge–fog–cloud computing

$R_t = P_t \cup Q_t$
(population combination of parent offloadingsolutions P_t and child offloading solutions Q_t)
Non-domination Pareto front $F = [F_1, F_2, \ldots]$
each task offloading solution $SP_p \in R_t$
$n_p \leftarrow 0$ // *n_p: Pareto domination counter*
$S_p \leftarrow \emptyset$ // *S_p: set of offloading solutions dominated by solution SP_p*
each task offloading solution $SP_q \in R_t$
$SP_p \prec SP_q$
$S_p \leftarrow S_p \cup \{SP_q\}$ // *If offloading solution SP_q is dominated by SP_p, add it to S_p*
$SP_q \prec SP_p$
$n_p \leftarrow n_p + 1$ // *Else increment the domination counter of SP_p*
$n_p == 0$
SP_p.rank $\leftarrow 1$ // *SP_p belongs to the first front*
$F_1 \leftarrow F_1 \cup \{SP_p\}$
$i \leftarrow 1$ // *Initialize the Pareto front counter*
$F_i \neq \emptyset$
$H \leftarrow \emptyset$ // *To register the following front members*
each offloading solution $SP_p \in F_i$
each offloading solution $SP_q \in S_p$
$n_q \leftarrow n_q - 1$
$n_q == 0$
SP_q.rank $\leftarrow i + 1$ // *SP_q is part of the next non-dominant front*
$H \leftarrow H \cup \{SP_q\}$
$i \leftarrow i + 1$
$F_i \leftarrow H$

- **The Crowding distance** Crowding distance is a metric employed in multi-objective optimization to assess the diversity of solutions within a population. It measures the distance between neighboring solutions in the objective space, encouraging solutions to be distributed evenly throughout the Pareto front. Solutions with higher crowding distances are considered more diverse and are given priority in the selection process. The aim of using crowding distance is to maintain a balance between convergence toward the Pareto front (i.e., the set of non-dominated solutions) and maintaining diversity in the solution set. In the context of

the Internet of Things (IoT), task offloading in a fog–cloud environment can be conceptualized as a set of potential offloading strategies for tasks. The objective of these strategies is to minimize latency and energy consumption. The crowding distance, which indicates the density of solutions around a particular offloading strategy, plays a crucial role in ensuring a diverse set of offloading strategies within the solution set. The algorithm 4 calculates the crowding distance for a set of non-dominated offloading strategies [10], denoted by *SP*, by considering each objective function. We consider a set of solution, denoted by *SP*, that require offloading. Each solution, represented by *SP*[*i*], has multiple objective functions to be optimized, including execution time and energy consumption. The objective is to maintain diversity in the solution set while optimizing these objectives. The task with the highest crowding distance is deemed to offer greater diversity in potential offloading options. The Pareto set assigns an infinite crowding distance to each solution *SP*[*i*], which is then ordered according to the objective function *m*. The boundary tasks are always selected, and the crowding distance is updated for each solution *SP*[*j*] from the second to the second-last solution in the sorted list. This normalization is achieved by dividing the value in question by the range of objective values within the Pareto set. This process prioritizes tasks with higher crowding distances, ensuring the implementation of diverse offloading strategies.

Algorithm 4 Crowding distance for IoT task offloading

Input : Pareto set of offloading solutions SP, m : Number of objective functions (energy, latency)
Output: Crowding distance values $SP[i]_{distance}$ for each offloading solution $SP[i]$
$L = |SP|$ // *Number of solutions*
for *i=1 to L* **do**
  $SP[i]_{distance} = 0$
for *Each objective m* **do**
  $SP = \mathrm{Sort}(SP, m)$
  $SP[1]_{distance} = \infty$
  $SP[L]_{distance} = \infty$
  for *j=2 to L-1* **do**
    $SP[j]_{distance} = SP[j]_{distance} + \frac{SP[j+1].m - SP[j-1].m}{SP_m^{max} - SP_m^{min}}$

- **Genetic operators**
- **The selection** technique is used to choose the chromosomes (solutions) used as input for the crossing procedures. In our model, we use a selection method that allows to select two individuals randomly. The concept is to randomly select two individuals in a population. The two selected individuals are given as parents to the crossing process.
- **The crossover** operator is applied to generate a new solution from two parent solutions. The objective of the crossover operator is to combine several parts of

individuals to create new individuals in the current population. After selecting the two parents (Parent1 and Parent2) from the current population, the crossover operator randomly selects two points in the scheduling order of the first parent. Then, we exchange the resource identifiers of the selected tasks with the same tasks in Parent2 [23]. In Fig. 3, we present an example of execution of the crossing on Parent1 and Parent2.

$$\text{Parent 1} = T_1(1), T_2(2), T_3(1), T_4(3), T_5(1), T_6(2)$$
$$\text{Parent 2} = T_1(1), T_2(1), T_3(2), T_4(2), T_5(3), T_6(3)$$

Firstly, the tasks T_2 and T_4 are randomly chosen from Parent1. Then, the VM segments are swapped between the two parents to form two child solutions Child1 and Child2.

Algorithm 5 Crossover

```
task list, VM list, parent solutions P1, P2
  child solution
  Random random = new Random()
  point = (int)
  (random.nextDouble() * length(task_list))
  Initialize child_solution
  for i = 1 to point do
child_solution[i] = P1[i]
  for i = point + 1 to length(task_list) do
child_solution[i] = P2[i]
  return child_solution
```

- **The mutation operator** In our simulation, the genetic mutation operator is a function that randomly alters a portion of the individual's chromosome (i.e., the encoded solution for the task offloading problem). This operator is used in the genetic algorithm (GA) framework to introduce diversity into the population and prevent premature convergence to a suboptimal solution. The choice of mutation operator depends on the specific problem being solved and the characteristics of the population. The mutation rate, which determines the probability of mutation occurring in each gene, is also an important parameter that needs to be care-

Table 3 Tasks/VMs correspondence solution table

Task	T_1	T_2	T_3	T_4	T_5	T_6
VM	VM_1^c	VM_2^f	VM_1^f	VM_2^c	VM_3^c	VM_3^f

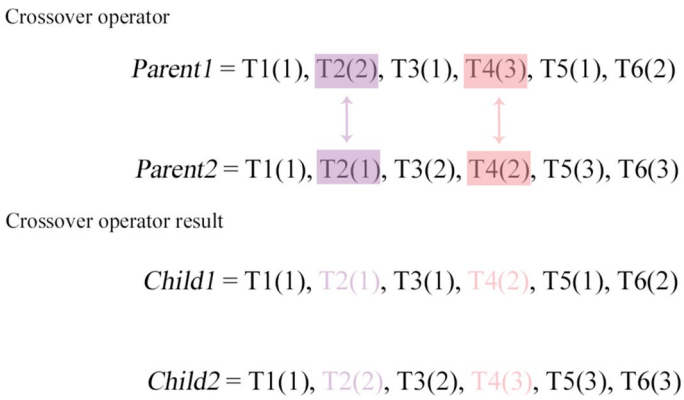

Fig. 3 Illustration of the crossing operator on *Parent*1 and *Parent*2 [23]

fully tuned to balance exploration and exploitation of the search space. The main point is to change the execution sequence of tasks in an individual. The adopted mutation operator has two main steps, (1) a task is selected randomly in the individual, (2) an alternative resource is selected randomly to replace the current task allocation [23]. Figure 4 shows the application of the permutation mutation method on the solutions *Child*1 and *Child*2 to generate two new children, namely *Child*1′ and *Child*2′. In the illustrated example, T_2 and T_4 were randomly selected and then permutation of the allocated resources was performed.

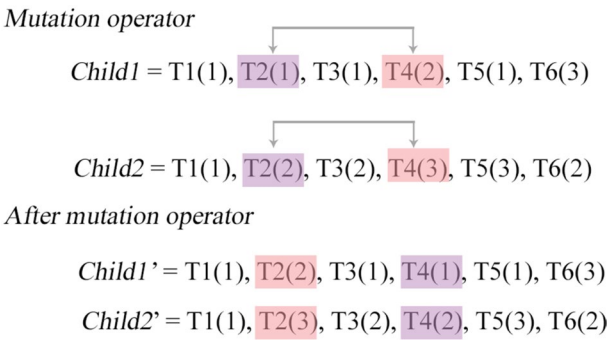

Fig. 4 Mutation operator illustration [23]

Algorithm 6 mutation

```
Input: task list ; VM list ;
Output: Solution task-Vm
Select a random mutation i Select a random mutation j
for i = 0 to tasks.size() do
    if (random.nextDouble() ≤ mutationRate) then
        solution = PT.getSolutions()
        VM = VM.get(random_n extInt(VMsize()))
        tasks.get(i).setResourceId(c.getId())
    _ _
```

6 Experimental study and results analysis

In this section, we present the different evaluations of our approach to prove its effectiveness.

The performance of the proposed approach was evaluated using iFogSim, an open-source simulator designed for modeling and simulating fog computing environments. Built-in metrics for evaluating energy consumption and latency, the primary objectives of the optimization approach, were provided by iFogSim, facilitating a thorough and accurate performance assessment. Furthermore, the toolkit allows for the creation of complex simulation models that incorporate various fog nodes, network configurations, and application models, thereby providing a comprehensive platform for evaluating the performance of fog computing systems [17]. The tests were performed on an Intel(R) Core(TM) i7-4800MQ CPU @ 2.70 GHz 2.70 GHz, installed RAM Memory 8.00, with a 64-bit Windows operating system, x64 processor.

6.1 Experimental settings

The parameters of the experiments include various IoT tasks, the fog and cloud computing environments, and the settings for the NSGA-II algorithm. The details of these settings can be found in Tables 4 and 6.

Table 4 System settings table

Settings	Value
Din	[100–3000KO]
w	[2500–3500 Megabit per second]
Ps	[0,5 KWatt]

6.1.1 NSGA-II algorithm settings

Following a series of tests, we have established the parameters for our NSGA-II algorithm. The specific settings adopted for the NSGA-II algorithm include:

- Number of Generations: 100.
- Population Size: 100.
- Mutation Rate: 0.1 (to minimize randomness).
- Crossover Rate: 0.5 (to enhance exploration of the search space).

These parameters were determined through iterative testing to optimize the performance of our algorithm. The number of generations and population size are crucial factors influencing the algorithm's convergence and search efficiency. Additionally, the mutation and crossover rates play a significant role in balancing exploration and exploitation of the search space. These settings were carefully selected to achieve a robust and effective performance of the NSGA-II algorithm in the context of our experiments.

6.1.2 Fog cloud environment settings

The simulated experimental environment employed in our work comprises 10 fog nodes collaborating with 5 cloud data centers. Both fog nodes and cloud data centers are outfitted with a diverse range of heterogeneous virtual machines (VMs). The specifications of these VMs draw inspiration from instances offered by Amazon EC2, a leading cloud computing service provider. The detailed specifications of the VM instances utilized in our experiments are enumerated in Table 6.

This experimental setup aims to emulate a realistic computing environment, reflecting the diversity and variability encountered in practical IoT, edge, and fog computing scenarios. By incorporating VMs with specifications akin to those provided by widely used cloud services, we enhance the relevance and applicability of our work to real-world cloud computing infrastructures. The variety of fog nodes and cloud data centers, along with their associated VM configurations, contributes to the comprehensiveness and robustness of our evaluation.

6.1.3 IoT tasks settings

In our experiments, we use a set of tasks with varying sizes randomly generated between [50–2000], simulating different types of IoT applications. Each task has an input data size randomly generated between [100–3000 KB]. The computational requirements for the tasks fall within the range of [1500–3500 KB]. All these characteristics are illustrated in Table 4

6.2 Performance assessment

The evaluation criteria primarily involve assessing the efficiency of our approach in generating an optimal offloading solution in terms of both energy consumption and latency. We opted to utilize the mean values of the results and employed Euclidean distance as a metric to compare and evaluate the optimality of our approach in comparison to two other approaches, referenced as [2] and [14]. The decision to employ this method was grounded in a comprehensive analysis of the data, with the expectation that it would yield accurate and reliable results.

To gauge the effectiveness of our approach, we compared it against the following two offloading strategies:

1. [2]: This approach proposes an ET-DTCO (Efficient Energy and Completion Time for Dependent Task Computation Offloading) algorithm. It considers two Quality of Service (QoS) parameters-efficient energy and completion time offloading for dependent tasks in Industry 4.0.
2. [14]: This approach formulates a joint energy consumption and latency optimization problem as a multi-objective problem. It addresses this problem using the non-dominant sorting genetic algorithm (NSGA-II) and the Bees Algorithm (BA).

Our evaluation focuses on the mean values of energy consumption and latency, and the comparison is conducted using Euclidean distance. This method of assessment was chosen after a thorough analysis of its suitability for our specific context, aiming to provide accurate and reliable insights into the performance of our approach relative to the other two approaches.

6.2.1 Optimal Pareto front: a trade-off analysis

Figure 5 illustrates the Pareto front for our system, showcasing a range of efficient solutions for offloading IoT tasks to the fog and cloud computing environment. This diagram highlights the trade-offs between average energy consumption and average

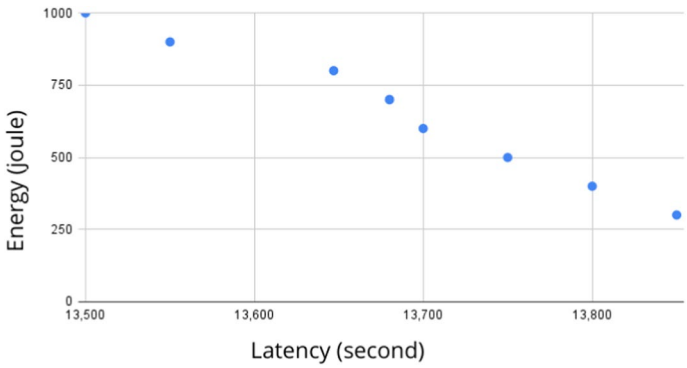

Fig. 5 Optimal Pareto front: a trade-off analysis

latency for users of the system. In this example, the experimental parameters include 20 fog VMs, 30 cloud VMs, and 100 tasks. The solutions are evaluated using two objective functions, f_1 and f_2, which measure the overall energy consumption and latency of IoT requests, respectively.

Our method successfully identifies optimal Pareto points along the front, helping users select solutions that align with their preferences for optimizing either energy consumption or latency. For example, selecting the first point on the Pareto front indicates a preference for minimizing energy consumption.

The Pareto front visually demonstrates the trade-offs between energy consumption and latency, allowing users to make informed decisions based on their specific priorities and requirements. The results achieved highlight the effectiveness of our approach in balancing these two critical metrics.

6.2.2 Energy consumption based evaluation

Figure 6 illustrates the comparative analysis of the total energy consumption of the system generated by our approach with other offloading strategies. The results demonstrate that the energy consumption of all approaches increases in conjunction with an increase in the number of IoT tasks. It is noteworthy that the approach designated as ET-DTCO exhibits the highest total energy consumption in comparison to the other methodologies. This can be attributed to the discharge of IoT requests without consideration of optimality conditions. Conversely, the approach designated as NSGAII-MMDE demonstrates a reduction in energy consumption in comparison to the

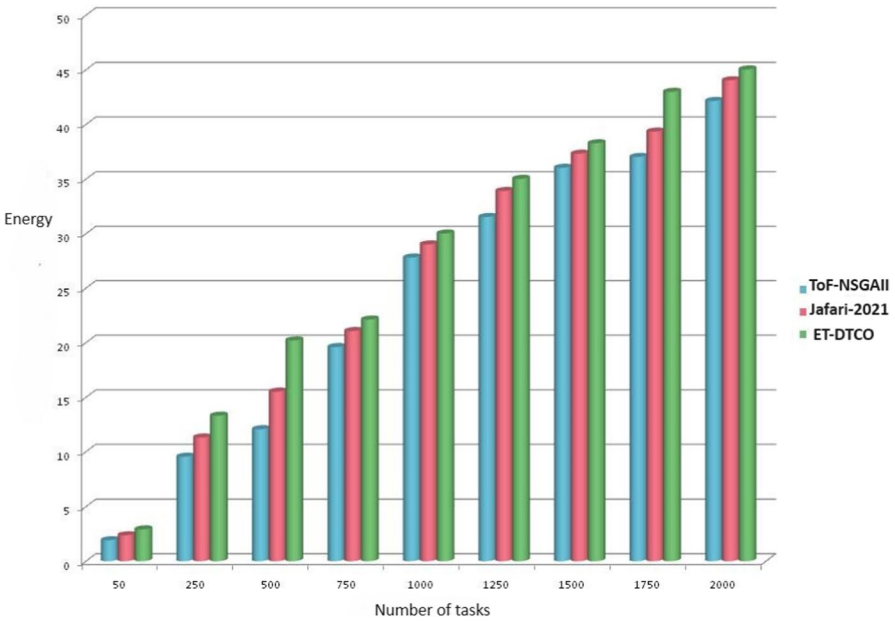

Fig. 6 Comparison of offloading performance in terms of global energy consumption of tasks

ET-DTCO approach, although it remains higher than that observed for our approach across all tested numbers of tasks. It is notable that our approach consistently generates task-offloading solutions that minimize energy consumption in comparison to the other approaches. These findings highlight the efficacy of our approach in optimizing energy usage, particularly as the number of IoT tasks increases. The comparison offers valuable insights into the relative performance of different offloading strategies in managing the energy consumption of the system.

In Fig. 7, we present the energy consumption values associated with tasks of varying sizes. This experiment aims to investigate how the size of a task influences the energy consumption of virtual machines (VMs). The results reveal a consistent trend across all tested approaches: as the size of a task increases, the energy consumed also increases. The size of a task denoted as T_i, ranges from 200 to 2000 KB. For our approach, the energy consumption increases gradually, not exceeding 5 MJ. Notably, when the size of the executed task is 600 KB, our TOF-NSGAII approach demonstrates an energy consumption of 1.2 MJ. This is lower than the energy consumed by the ET-DTCO approach, which registers 1.3 MJ, and also lower than the NSGAII-MMDE approach by 1.41 MJ. Figure 7 consistently illustrates that our approach consumes less energy than the other tested approaches across different task sizes. This observation underscores the efficiency of the proposed TOF-NSGAII offloading strategy. It indicates that our approach is particularly effective in managing energy consumption, even as the size of tasks varies, making it a robust solution for diverse IoT workloads.

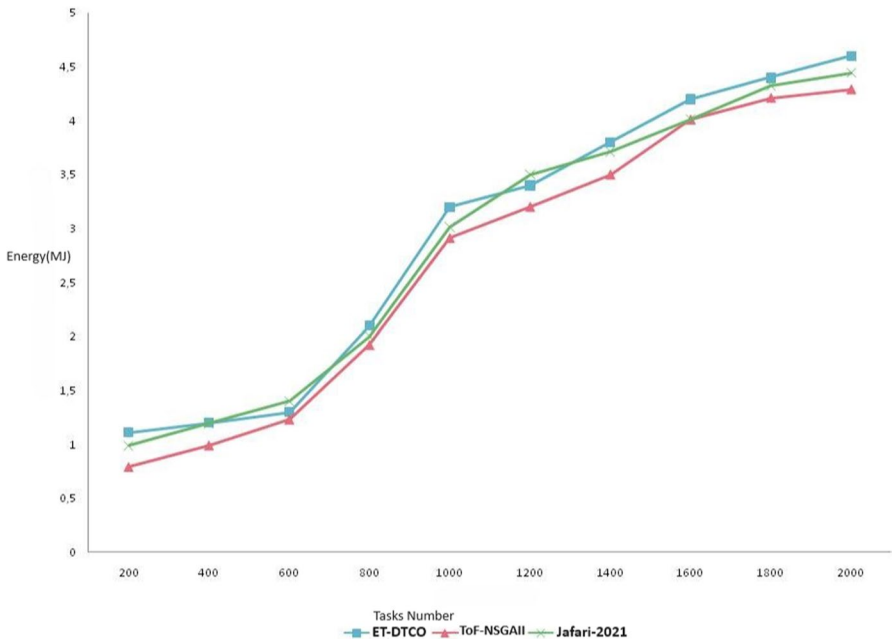

Fig. 7 Energy consumption with different job sizes

6.2.3 Latency based evaluation

To assess our approach in terms of multi-objective optimization, we conducted a comparison of the latency in the generated solutions with other approaches. Figure 8 illustrates the latency results obtained from the offloading solutions of a task set ranging from 50 to 2000 tasks. The results consistently show that our TOF-NSGAII approach outperforms other approaches across different numbers of tasks, with a negligible difference. For example, with 50 tasks, our approach provides an offloading solution with a latency value of 51.9 s. In comparison, the NSGAII-MMDE approach and the ET-DTCO approach yield latencies of 51 s and 51.5 s, respectively. Notably, the observed differences between the latency values generated by our approach and others range from [0.4–0.9 s], which is not a significant margin. This indicates that our approach consistently produces offloading solutions that are efficient in terms of latency. The results presented in Fig. 8 affirm the capability of our TOF-NSGAII approach to generate solutions that strike a favorable balance between energy consumption and latency across a range of task quantities. The minimal differences in latency further emphasize the effectiveness of our approach in achieving multi-objective optimization for offloading IoT tasks.

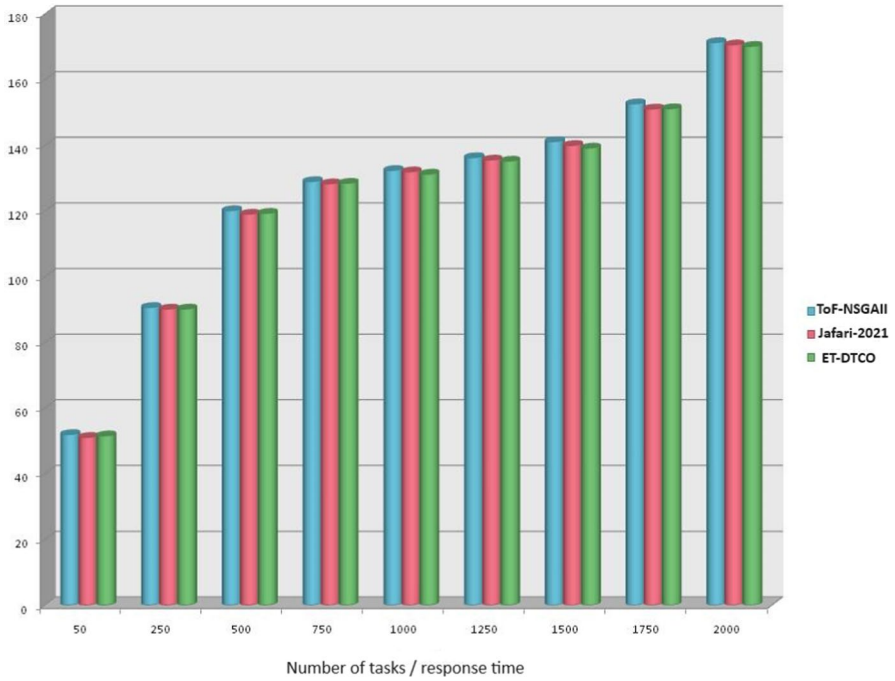

Fig. 8 Comparison of the global latency results of the different approaches

6.2.4 Offloading rate

In this experiment, we tested the computational capacity of the fog VMs by incrementally increasing the number of tasks, starting from 50 tasks, using iFogSim for simulation. Our observations indicated that the fog VMs could handle up to 1000 tasks. However, as the number of tasks increased, the computational demands also grew, necessitating more processing power and data transfer. To illustrate this, we present a graph showing the relationship between the number of tasks and the offloading rate to the cloud. Ultimately, due to the limited computational capacity of the fog, additional resources are required. In Fig. 9, we demonstrate the offloading performance for different numbers of tasks to the cloud and fog. The results show that the percentage of tasks offloaded to the cloud is not significant until the number of tasks becomes high. For example, the offload rate in the fog environment is higher than in the cloud computing environment, with a percentage ranging from 30% to 70% for the number of IoT tasks between 50 and 1000. However, this percentage of offloading to cloud computing increases with the number of tasks, which affects latency. Based on these findings, our approach will focus on minimizing the number of tasks offloaded to the cloud to optimize latency.

6.2.5 Multi-objective optimization improvement

In Table 5, we present the performance of our approach in terms of multi-objective optimization, along with a measure of improvement. The results obtained by

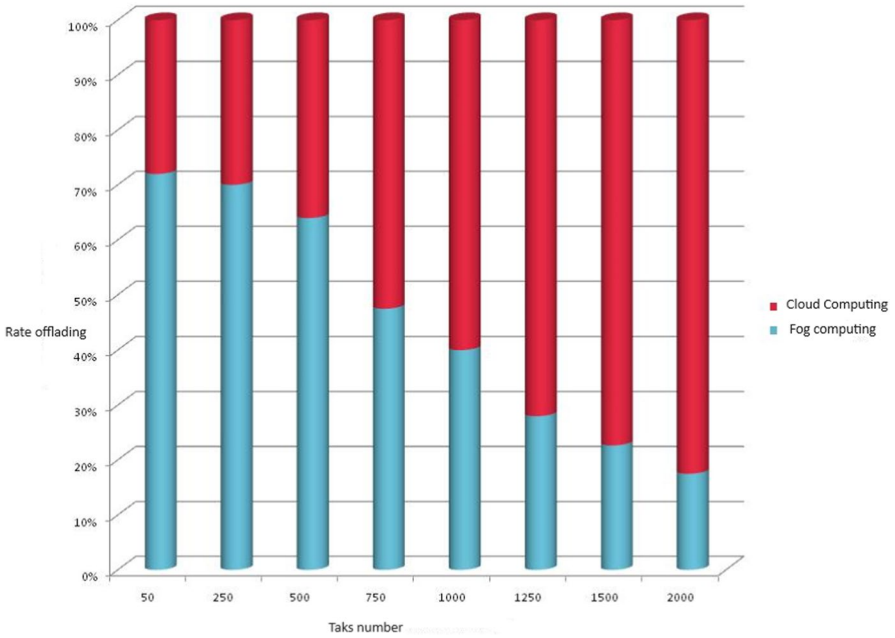

Fig. 9 Tasks offloading percentage in fog and cloud environments

Table 5 Comparison of results and improvement of approaches in terms of latency and energy consumption to offload different numbers of IoT tasks

Number of tasks	Latency (s)			Energy (MJ)			Improvement (IM)/%			
	TOF-NSGAII (1)	NSGAII-MMDE (2)	ET-DTCO (3)	TOF-NSGAII (4)	NSGAII-MMDE (5)	ET-DTCO (6)	IM NSGAII-MMDE latency (1)–(2)/(1)	IM TOF-NSGAII energy (5)–(4)/(5)	IM ET-DTCO Latency (1)–(3)/(1)	IM TOF-NSGAII energy (6)–(4)/(6)
50	51,9	51	51,5	1,90625	2,036	2,91	1,73%	6,4%	0,008%	34,4 %
250	90,6	90,1	90,12	9,53	11.3	13,3	0,55 %	15,7%	0,005%	28,3%
500	120,12	119,02	119,3	12,06	15,52	20,2	0,92%	20,3%	0,007%	8,2%
750	129,021	128,32	128,48	19,59	21,06	22,12	0,54%	6,98%	0,004%	11,43%
1000	132,32	131,9	131,19	27,8	29,1	30,01	0,32%	3,8%	0,009	7,3%
1250	136,22	135,5	135,15	31,53	33,9	35,02	0,53%	18,5%	0,008%	9,96 %
1500	141,61	139,96	139,159	36,0365	37,3025	38,2569	0,82%	10,6%	0,0139%	5,8%
1750	152,61	151,02	151,102	37,0025	39,3321	42,962	1,04%	7,9%	0,010%	13,87%
2000	171,336	170,62	171,013	42,1258	44,0251	45,025	0,42%	8%	0,0019%	6,44%

Table 6 characteristic of VMs

Environment	VM instances	Memory GB	VCPU	Networks/GB
Fog computing	t3.small	2	1	Up to 5
	t3.xlarge	16	4	Up to 5
Cloud computing	c4.large	3.75	8	XX
	c4.xlarge	7.5	16	Haut height

the SGAII-MMDE, ET-DTCO, and TOF-NSGAII approaches are represented in Figs. 8 and 6. In Fig. 6, the average system energy consumption of NSGAII-MMDE and ET-DTCO is higher than that of our TOF-NSGAII approach. Conversely, Fig. 8 shows that the average latency generated by the NSGAII-MMDE and ET-DTCO approaches is lower than that of our TOF-NSGAII approach. The results indicate that the offloading solutions obtained by the NSGAII-MMDE and ET-DTCO approaches are slightly more optimal in terms of latency compared to those generated by TOF-NSGAII. For instance, in the case of 50 tasks, the NSGAII-MMDE approach achieves a latency of 51 s, which is 1.73% lower than the 51.9 s obtained by TOF-NSGAII. Similarly, the ET-DTCO approach achieves a latency of 51.5 s, which is 0.77% lower than our approach. Across different numbers of tasks, the latency improvement of the NSGAII-MMDE and ET-DTCO approaches over our approach did not exceed 2%. However, our TOF-NSGAII approach demonstrates superior performance in optimizing energy consumption. For all numbers of IoT tasks, the energy efficiency improvement achieved by the TOF-NSGAII approach reaches 20.3%. This suggests that our approach successfully obtains optimal IoT task offloading solutions that meet the specified objectives and remain non-dominant when compared to the solutions generated by the ET-DTCO and NSGAII-MMDE approaches.

7 Conclusion and future trends

The focus of this work is on the problem of application offloading in the fog cloud environment, taking into account the unique characteristics of the Internet of Things (IoT) environment. The proposed approach includes a decision phase that models the data emitted by IoT devices and evaluates them to determine the most appropriate offloading strategy. To achieve this, we introduced the TOF-NSGAII approach, which is designed to minimize energy consumption and latency when offloading IoT tasks in a fog cloud environment. Each IoT object sends a set of tasks, each with specific characteristics such as input data size, and our algorithm assigns tasks to specific virtual machines to optimize resource utilization. By leveraging the collaborative resources of both fog and cloud computing, the TOF-NSGAII approach is able to achieve its objectives of minimizing energy consumption and latency, as demonstrated by the results of our experiments.

This paper introduces the TOF-NSGAII approach, which aims to solve the IoT task offloading problem in the fog cloud computing environment. The approach is designed to optimize multiple quality of service (QoS) metrics while establishing collaboration between fog and cloud computing resources by minimizing latency and energy consumption costs. To achieve this, we developed a robust model that represents the various entities involved in the offloading process and leverages a powerful strategy to achieve the desired mission within the given constraints. We used the NSGA-II genetic algorithm to obtain a set of non-dominated solutions through selection, crossover, mutation, and crowding distance principles. This work provides a foundation for future research in this area, and we plan to expand on this work by further developing the following aspects:

- Aggregation approach for a multi-objective optimization offloading solution.
- The problem of real-time task offloading with the dynamics of user objectives and preferences; also takes into account the dynamic nature of the fog environment.
- The objective is to enhance the solution through collaboration with reinforcement learning algorithms, thereby facilitating continuous learning and adaptation in real-time environments. This approach renders them potentially more suited for dynamic and real-time optimization tasks in fog computing environments.

References

1. Aazam M et al (2020) Cloud of things (CoT): cloud-fog-IoT task offloading for sustainable internet of things. IEEE Trans Sustain Comput 7(1):87–98
2. Abdel-Kader RF, El-Sayad NE, Rizk RY (2021) Efficient energy and completion time for dependent task computation 4.0 offloading algorithm in industry. PLoS ONE 16(6):e0252756
3. Alasmari MK, Alwakeel SS, Alohali YA (2023) A multi-classifiers based algorithm for energy efficient tasks offloading in fog computing. Sensors 23(16):7209
4. Alfakih T et al (2020) Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA. IEEE Access 8:54074–54084
5. AlShathri SI, Chelloug SA, Hassan DSM (2022) Parallel meta-heuristics for solving dynamic offloading in fog computing. Mathematics 10(8):1258
6. Baek J, Kaddoum G (2020) Heterogeneous task offloading and resource allocations via deep recurrent reinforcement learning in partial observable multifog networks. IEEE Internet Things J 8(2):1041–1056
7. Cirillo R (2012) The economics of Vilfredo Pareto. Routledge, London
8. Cloud H (2011) The nist definition of cloud computing. Natl Inst Sci Technol Spec Publ 800:145
9. Cui K et al (2019) Learning-based task offloading for marine fog-cloud computing networks of USV cluster. Electronics 8(11):1287
10. Deb K et al (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 6(2):182–197
11. El Idrissi M, Elbeqqali O, Jamal R (2019) A review on relationship between Iot-cloud computing-fog computing (applications and challenges). In: (2019) third international Conference on Intelligent Computing in Data Sciences (ICDS). IEEE, pp 1–7
12. Haghnegahdar L, Joshi SS, Dahotre NB (2022) From IoT-based cloud manufacturing approach to intelligent additive manufacturing: industrial internet of things—an overview. Int J Adv Manuf Technol 1–18
13. Hussein MK, Mousa MH (2020) Efficient task offloading for IoT-based applications in fog computing using ant colony optimization. IEEE Access 8:37191–37201

14. Jafari V, Rezvani MH (2021) Joint optimization of energy consumption and time delay in IoT-fog-cloud computing environments using NSGA-II metaheuristic algorithm. J Amb Intell Human Comput 1–24
15. Jazayeri F, Shahidinejad A, Ghobaei-Arani M (2021) Autonomous computation offloading and auto-scaling the in the mobile fog computing: a deep reinforcement learning-based approach. J Ambient Intell Humaniz Comput 12(8):8265–8284
16. Keshavarznejad M, Rezvani MH, Adabi S (2021) Delay-aware optimization of energy consumption for task offloading in fog environments using metaheuristic algorithms. Clust Comput 24(3):1825–1853
17. Khan EUY, Soomro TR, Brohi MN (2022) iFogSim: a tool for simulating cloud and fog applications. In: 2022 International Conference on Cyber Resilience (ICCR). IEEE, pp 01–05
18. Kumari N, Yadav A, Jana PK (2022) Task offloading in fog computing: a survey of algorithms and optimization techniques. Comput Netw 214:109137
19. Laroui M et al (2021) Edge and fog computing for IoT: a survey on current research activities and future directions. Comput Commun 180:210–231
20. Li G et al (2019) Energy consumption optimization with a delay threshold in cloud-fog cooperation computing. IEEE Access 7:159688–159697
21. Liu J et al (2022) Auction-based dependent task offloading for IoT users in edge clouds. IEEE Internet Things J 10(6):4907–4921
22. Liu J et al (2024) Task graph offloading via deep reinforcement learning in mobile edge computing. Futur Gener Comput Syst 158:545–555
23. Mokni M et al (2022) Cooperative agents-based approach for workflow scheduling on fog-cloud computing. J Ambient Intell Humaniz Comput 13(10):4719–4738
24. Mukherjee M, et al (2020) Distributed deep learning-based task offloading for UAV-enabled mobile edge computing. In: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS). IEEE, pp 1208–1212
25. Munene KI et al (2022) A throughput drop estimation model and its application to joint optimization of transmission power, frequency channel, and channel bonding in IEEE 802.11 n WLAN for large-scale IoT environments. Internet of Things 20:100583
26. Nittala SSS (2022) Service innovation enabled by Internet of Things and cloud computing-a service-dominant logic perspective. Technol Anal Strateg Manag 34(4):433–446
27. Peng G et al (2021) Constrained multiobjective optimization for IoT-enabled computation offloading in collaborative edge and cloud computing. IEEE Internet Things J 8(17):13723–13736
28. Rahbari D, Nickray M (2020) Task offloading in mobile fog computing by classification and regression tree. Peer-to-Peer Netw Appl 13:104–122
29. Satyanarayanan M, et al (2021) The role of edge offload for hardware-accelerated mobile devices. In: Proceedings of the 22nd international workshop on mobile computing systems and applications, pp 22–29
30. Shahidinejad A, Ghobaei-Arani M (2022) A metaheuristic-based computation offloading in edge-cloud environment. J Ambient Intell Humaniz Comput 13(5):2785–2794
31. Shahryari O-K et al (2021) Energy and task completion time trade-off for task offloading in fog-enabled IoT networks. Pervasive Mob Comput 74:101395
32. Shreyas J et al (2020) Application of computational intelligence techniques for internet of things: an extensive survey. Int J Comput Intell Stud 9(3):234–288
33. Singh R, Gehlot A, Sharma D (2022) Futuristic sustainable energy and technology. CRC Press, New York
34. Sofla MS et al (2022) Towards effective offloading mechanisms in fog computing. Multimedia Tools Appl 81(2):1997
35. Tran-Dang H, Kim D-S (2023) Cooperation for distributed task offloading in fog computing networks. In: Cooperative and distributed intelligent computation in fog computing: concepts, architectures, and frameworks, pp 33–45
36. Tran-Dang H, Kim D-S (2023) Dynamic collaborative task offloading for delay minimization in the heterogeneous fog computing systems. J Commun Netw
37. Tran-Dang H, Kim D-S (2021) FRATO: fog resource based adaptive task offloading for delay-minimizing IoT service provisioning. IEEE Trans Parallel Distrib Syst 32(10):2491–2508
38. Vemireddy S, Rout RR (2021) Fuzzy reinforcement learning for energy efficient task offloading in vehicular fog computing. Comput Netw 199:108463

39. Wang K et al (2019) Learning-based task offloading for delay-sensitive applications in dynamic fog networks. IEEE Trans Veh Technol 68(11):11399–11403
40. Yadav J, et al (2023) E-MOGWO algorithm for computation offloading in fog computing. In: Intelligent automation and soft computing 36.1
41. You Q, Tang B (2021) Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. J Cloud Comput 10(1):1–11

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Ibtissem Mokni[1,2] · Sonia Yassa[2]

✉ Ibtissem Mokni
ibtyssemmokni@gmail.com

Sonia Yassa
sonia.yassa@cyu.fr

[1] Modeling of Automated Reasoning Systems laboratory (MARS), University of Sousse, Sousse, Tunisia

[2] The Information Processing and Systems Teams Laboratory (ETIS), CY Cergy Paris University, Cergy, France